

Entrega - Fase III

Integrantes

- Gabriel Teotonio de Castro Coelho Costa
- Thales Duque Camara
- Tiago Delgado Rocha
- Henrique Amorim Soares
- Lucas Gontijo Riani

Parte 1 - Documentacao Tecnica do Aplicativo

Visao geral

O sistema GestaoSaude foi desenvolvido em Java, sem uso de SGBD, utilizando persistencia direta em arquivos binarios. A interface do usuario e web, servida pelo Javalin, e nao depende de console para as operacoes desta fase.

As principais entidades persistidas sao:

- **Usuario**
- **Paciente**
- **Consulta**
- **Procedimento**
- **ConsultaProcedimento**

Os dados ficam na pasta **data/**, enquanto os indices ficam em **data/indices/**.

Padrao de persistencia

Cada tabela possui um arquivo proprio em disco. Os registros seguem o mesmo padrao:

- cabeçalho com ultimo ID utilizado
- ponteiro para lista de espacos livres
- lapide
- tamanho do registro
- carga util serializada em bytes

A exclusao e logica. Quando um registro e removido, sua lapide e marcada, e o espaco liberado pode ser reaproveitado por novas insercoes.

Indices primarios

As entidades principais usam indice primario baseado em Hash Extensivel. O indice mapeia:

- chave: **id**
- valor: endereco do registro no arquivo **.db**

Com isso, a busca por ID nao depende de varredura sequencial do arquivo.

Relacionamento 1:N

O relacionamento **Paciente 1:N Consulta** foi mantido com Hash Extensível e lista invertida encadeada.

O índice usa:

- chave: **idPaciente**
- valor: ponteiro para a lista de consultas do paciente

Assim, o sistema consegue listar consultas de um paciente sem percorrer todo o arquivo de consultas.

Relacionamento N:N

O relacionamento **Consulta N:N Procedimento** foi implementado pela tabela intermediária **ConsultaProcedimento**.

Essa tabela possui:

- **idConsulta**
- **idProcedimento**
- **observacao**

A chave primária lógica da tabela intermediária é composta por:

- **idConsulta**
- **idProcedimento**

Para manter compatibilidade com a classe genérica **Arquivo<T>**, o registro ainda possui um **id** interno. Esse **id** serve para localizar fisicamente o registro, mas a regra de unicidade do relacionamento é garantida pelo índice de chave composta.

Índices do relacionamento N:N

Foram criados três índices para **ConsultaProcedimento**:

- Hash Extensível da chave composta **idConsulta + idProcedimento**
- Hash Extensível por **idConsulta**
- Hash Extensível por **idProcedimento**

Os índices por **idConsulta** e por **idProcedimento** apontam para listas invertidas em disco. Isso permite navegar pelo relacionamento nos dois sentidos:

- listar procedimentos de uma consulta
- listar consultas que contêm determinado procedimento

Consulta ordenada com Árvore B+

A Árvore B+ foi usada na tabela **Procedimento**, na funcionalidade de listar procedimentos em ordem alfabética pelo nome do exame.

O índice fica em:

- **data/indices/procedimentos_nome.bplus**

A rota usada pela interface e:

- `/procedimentos/ordenado/nome`

Nessa consulta, o sistema percorre as folhas da Arvore B+ em ordem e recupera os procedimentos pelos IDs armazenados no indice, sem ordenar a lista em memoria principal no momento da consulta.

Integridade referencial

O sistema aplica as seguintes regras:

- nao cria consulta para paciente inexistente
- nao cria consulta para usuario inexistente
- nao cria vinculo `ConsultaProcedimento` para consulta inexistente
- nao cria vinculo `ConsultaProcedimento` para procedimento inexistente
- nao permite duplicidade da chave composta `idConsulta + idProcedimento`
- ao excluir consulta, remove os vinculos dela com procedimentos
- nao permite excluir procedimento que ainda esteja vinculado a consultas
- ao atualizar um vinculo, remove a chave antiga dos indices e grava a nova

Interface web

A interface web permite operar o relacionamento sem console:

- na tela de consultas, e possivel vincular procedimentos a uma consulta
- na visualizacao de consulta, e possivel listar os procedimentos vinculados
- na tela de procedimentos, e possivel listar as consultas que usam determinado procedimento
- na tela de procedimentos, existe uma opcao para listar em ordem alfabetica usando Arvore B+

Parte 2 - Formulario Tecnico da Fase III

1. Qual foi o relacionamento N:N escolhido e quais tabelas ele conecta?

O relacionamento `N:N` escolhido foi `Consulta N:N Procedimento`. Ele conecta a tabela de consultas com a tabela de procedimentos por meio da tabela intermediaria `ConsultaProcedimento`.

Uma consulta pode possuir varios procedimentos, e um mesmo procedimento pode aparecer em varias consultas.

2. Qual estrutura de indice foi utilizada (B+ ou Hash Extensivel)? Justifique a escolha.

No relacionamento `N:N`, foi utilizado Hash Extensivel, pois as principais operacoes sao buscas diretas por igualdade de chave: buscar procedimentos de uma consulta, buscar consultas de um procedimento e verificar se a chave composta ja existe.

A Arvore B+ tambem foi usada no sistema em uma funcionalidade de consulta ordenada: listagem de procedimentos por nome. Ela foi escolhida para essa funcionalidade porque permite recuperar os registros em ordem percorrendo as folhas do indice, sem ordenar em memoria principal durante a consulta.

3. Como foi implementada a chave composta da tabela intermediaria?

A chave composta e formada por `idConsulta` e `idProcedimento`.

No indice, os dois valores inteiros sao combinados em uma chave `long`:

```
((long) idConsulta << 32) | (idProcedimento & 0xffffffffL)
```

Essa chave e usada pelo indice `consulta_procedimento_pk_composta`, garantindo que a mesma combinacao nao seja cadastrada mais de uma vez.

4. Como e feita a busca eficiente de registros por meio do indice?

Para buscar um vinculo especifico, o sistema calcula a chave composta e consulta o Hash Extensivel. O resultado aponta para o `id` interno do registro associativo, que e usado para recuperar o registro no arquivo.

Para navegar pelos dois lados do relacionamento, existem indices auxiliares:

- `consulta_procedimento_por_consulta`
- `consulta_procedimento_por_procedimento`

Cada um aponta para uma lista invertida em disco, evitando varredura completa no arquivo da tabela intermediaria.

5. Como o sistema trata a integridade referencial (remocao/atualizacao) entre as tabelas?

Antes de criar um vinculo, o sistema verifica se a consulta e o procedimento existem. Tambem verifica se a chave composta ja esta cadastrada.

Ao excluir uma consulta, seus vinculos com procedimentos sao removidos. Ao excluir um procedimento, o sistema verifica se ainda existem consultas vinculadas; caso existam, a exclusao e bloqueada.

Na atualizacao de um vinculo, os indices antigos sao removidos e os novos sao inseridos, mantendo a sincronizacao entre dados e indices.

6. Como foi organizada a persistencia dos dados dessa nova tabela (mesmo padrao de cabecalho e lapide)?

A tabela intermediaria usa o mesmo padrao das demais tabelas do sistema.

O arquivo principal e:

- `data/consulta_procedimentos/consulta_procedimentos.db`

Ele possui cabecalho, registros com lapide, tamanho do registro e dados serializados. Na exclusao, o registro recebe lapide, e seu espaco pode ser reaproveitado posteriormente.

7. Descreva como o codigo da tabela intermediaria se integra com o CRUD das tabelas principais.

A classe `ConsultaProcedimentoDAO` centraliza as operacoes da tabela intermediaria.

No CRUD de consultas, os procedimentos selecionados na interface são gravados como vínculos em `ConsultaProcedimento`. Ao visualizar uma consulta, o sistema usa o índice por `idConsulta` para recuperar os procedimentos vinculados.

No CRUD de procedimentos, cada procedimento possui uma ação para listar as consultas em que aparece. Essa listagem usa o índice por `idProcedimento`.

Ao excluir uma consulta, os vínculos correspondentes são removidos. Ao excluir um procedimento, o sistema impede a remoção caso existam consultas vinculadas.

8. Descreva como esta organizada a estrutura de diretórios e módulos no repositório após esta fase.

A estrutura ficou organizada assim:

- `src/main/java/model`: modelos e serialização dos registros
- `src/main/java/dao`: DAOs, persistência, Hash Extensível, listas invertidas, índice composto e Árvore B+
- `src/main/java/principal`: servidor Javalin e rotas HTTP
- `src/main/resources/public`: interface web
- `data`: arquivos binários das tabelas
- `data/indices`: arquivos dos índices persistidos
- `docs`: documentação e arquivo de entrega

Arquivos principais desta fase:

- `ConsultaProcedimentoDAO.java`
- `IndiceConsultaProcedimento.java`
- `HashExtensivelLongLong.java`
- `ArvoreBMaisProcedimentoNome.java`
- `ProcedimentoDAO.java`
- `Main.java`

Parte 3 - Links

- GitHub: <https://github.com/gabrielteotonio10/AEDSIII-GestaoSaude>